

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH & TRUYỀN THÔNG
BỘ MÔN AN TOÀN THÔNG TIN

NGUYỄN VIỆT TÙNG – 23521746

NGUYỄN LỘC TỶ – 23521757

BÁO CÁO ĐỒ ÁN

**Triển khai Federation & SSO an toàn trên môi trường
Multi-Cloud**

**Secure Implementation of Federation & SSO in a Multi-Cloud
Environment**

GIẢNG VIÊN HƯỚNG DẪN

TS. NGUYỄN NGỌC TỰ

TP. HỒ CHÍ MINH, 2026

LỜI CẢM ƠN

Các thành viên trong nhóm 14 muốn gửi lời cảm ơn chân thành sâu sắc tới giảng viên hướng dẫn TS. Nguyễn Ngọc Tự đã tận tình hỗ trợ nhóm để hoàn thành đồ án này. Qua đồ án này, nhóm đã tích lũy được nhiều kiến thức bổ ích về chuyên môn cũng như rèn luyện kỹ năng làm việc nhóm. Mặc dù các thành viên trong nhóm đã cố gắng hoàn thành đồ án một cách hoàn thiện nhất nhưng do thiếu sót về kinh nghiệm cũng như một số hạn chế khác nên đồ án này cũng không thể tránh khỏi các sai sót. Nhóm rất mong nhận được sự cảm thông, chia sẻ và góp ý từ thầy. Nhóm 14 xin chân thành cảm ơn!

MỤC LỤC

Chương 1. TỔNG QUAN ĐỀ TÀI.....	2
1.1. Bối cảnh và thực trạng	2
1.2. Lý do chọn đề tài.....	2
1.3. Mục tiêu đề tài.....	3
1.3.1. Mục tiêu kỹ thuật.....	3
1.3.2. Mục tiêu bảo mật và vận hành	3
1.4. Phạm vi đề tài.....	4
1.4.1. Đối tượng nghiên cứu	4
1.4.2. Phạm vi đề tài	4
1.5. Phương pháp thực hiện	4
1.6. Ý nghĩa khoa học và thực tiễn.....	5
1.6.1. Ý nghĩa thực tiễn	5
1.6.2. Ý nghĩa kỹ thuật.....	5
Chương 2. CƠ SỞ LÝ THUYẾT VÀ NỀN TẢNG.....	6
2.1. Tổng quan về định danh số (Digital Identity) và IAM	6
2.1.1. Khái niệm định danh số.....	6
2.1.2. IAM là gì?	6
2.2. Xác thực và phân quyền trong hệ thống Cloud	7
2.2.1. Xác thực (Authentication).....	7
2.2.2. Phân quyền (Authorization).....	7
2.3. Bài toán định danh trong Multi-Cloud	8
2.3.1. Credential Sprawl (bùng nổ thông tin đăng nhập).....	8
2.3.2. Chính sách truy cập không đồng nhất	8

2.3.3.	Khó giám sát và điều tra sự cố.....	8
2.4.	Single Sign-On (SSO) và lợi ích trong hệ thống Cloud	9
2.4.1.	Khái niệm SSO	9
2.4.2.	Lợi ích của SSO	9
2.4.3.	Rủi ro của SSO	9
2.4.4.	Các giao thức triển khai SSO phổ biến.....	10
2.4.4.1.	SAML 2.0	10
2.4.4.2.	OAuth2	11
2.4.4.3.	OpenID Connect (OIDC)	11
2.4.4.4.	So sánh tổng quát SAML và OIDC	13
2.5.	Federation (liên kết định danh) trong kiến trúc hiện đại	13
2.5.1.	Khái niệm Federation.....	13
2.5.2.	SSO và Federation khác nhau thế nào?	13
2.6.	Ánh xạ định danh và phân quyền (Identity Mapping & Role Mapping)....	14
2.6.1.	Mapping danh tính	14
2.6.2.	Mapping quyền: nguyên tắc và sai lầm thường gặp.....	14
2.7.	Các vấn đề bảo mật thường gặp trong SSO/Federation.....	14
2.7.1.	Replay Attack	14
2.7.2.	Token Theft / Session Hijacking.....	15
2.7.3.	Signature Bypass / Unsigned Token	15
2.7.4.	Misconfiguration.....	15
2.7.5.	Privilege Escalation	15
2.8.	Hardening cho hệ thống định danh tập trung.....	16
2.8.1.	Hardening ở Identity Provider (IdP)	16

2.8.2.	Hardening ở Service Provider (SP).....	16
2.8.3.	Hardening token và khóa ký.....	16
2.9.	Logging – Monitoring – Auditing trong hệ thống Cloud.....	16
2.9.1.	Vì sao cần giám sát?.....	16
2.9.2.	Log cần thu thập.....	17
2.9.3.	Chỉ số và dấu hiệu bất thường (Indicators)	17
Chương 3.	KIẾN TRÚC & TRIỂN KHAI THỰC TẾ.....	18
3.1.	Kiến trúc giải pháp	18
3.2.	Kiến trúc triển khai.....	19
3.3.	Triển khai Identity Provider & Service Provider.....	20
3.3.1.	Cài đặt và thiết lập môi trường.....	20
3.3.2.	Cấu hình Realm và Federation cho OpenStack (OIDC).....	20
3.3.3.	Cấu hình Federation cho AWS Identity Center (SAML).....	21
3.4.	Triển khai ELK Pipeline với TLS.....	21
3.4.1.	Thiết lập hạ tầng ELK.....	21
3.4.2.	Hardening.....	21
3.4.2.1.	Certificate Management	22
3.4.2.2.	Encryption in Transit.....	22
3.4.2.3.	Authentication & RBAC	22
3.5.	Logstash ETL Pipeline.....	23
3.5.1.	Input Layer.....	23
3.5.2.	Process Layer	23
3.5.3.	Enrichment & Output.....	24
3.6.	Kibana Detection Rule: Impossible Travel	26

3.6.1.	Xây dựng dữ liệu lịch sử (Transform).....	26
3.6.2.	Làm giàu dữ liệu thời gian thực (Enrich Policy).....	26
3.6.3.	Tính toán vận tốc di chuyển (Scripted Field)	26
3.7.	Ansible Playbook: Revoke Session	27
3.8.	Mục tiêu.....	28
Chương 4. KẾT QUẢ VÀ NHẬN XÉT		29
4.1.	Functionality Test	29
4.2.	MFA Enforcement.....	29
4.3.	Unsigned Token.....	29
4.4.	Alert: Impossible Travel.....	30
4.5.	Replay Attack.....	30
4.6.	Nhận xét	31
4.7.	Hướng phát triển.....	32

TÓM TẮT ĐỒ ÁN

Đồ án tập trung nghiên cứu và triển khai hệ thống quản trị định danh tập trung trên môi trường Multi-cloud (AWS và OpenStack), giải quyết bài toán credential sprawl và rủi ro an ninh trong hạ tầng Hybrid Cloud.

Các thành phần cốt lõi của giải pháp bao gồm:

- Hệ thống Federation & SSO: Sử dụng Keycloak đóng vai trò là Identity Provider (IdP) trung tâm, thiết lập cơ chế SSO thông qua giao thức OIDC (cho OpenStack) và SAML 2.0 (cho AWS). Hệ thống triển khai mô hình Global STS và Token Exchange Service để nhất quán hóa trải nghiệm người dùng.
- Giám sát và Phân tích: Xây dựng ETL Pipeline trên nền tảng ELK Stack để thu thập và chuẩn hóa nhật ký xác thực từ các nguồn không đồng nhất (Keycloak, Keystone, CloudTrail). Toàn bộ hạ tầng giám sát được thắt chặt bảo mật (Hardening) bằng TLS/mTLS và cơ chế RBAC.
- Tự động hóa phản ứng (SOAR): Triển khai quy tắc phát hiện tấn công "Impossible Travel". Hệ thống sử dụng ngôn ngữ Painless và công thức Haversine để tính toán vận tốc di chuyển giữa các lần đăng nhập. Khi phát hiện bất thường, Ansible Playbook sẽ tự động được kích hoạt để thu hồi phiên truy cập (Revoke Session) của người dùng ngay lập tức.

Kết quả thực nghiệm: Hệ thống đã chứng minh được khả năng vận hành ổn định trong việc hợp nhất định danh, thực thi chính sách MFA, và có khả năng chống lại các phương thức tấn công phổ biến như Replay Attack hay Unsigned Token.

Chương 1. TỔNG QUAN ĐỀ TÀI

1.1. Bối cảnh và thực trạng

Trong những năm gần đây, xu hướng chuyển đổi số khiến các doanh nghiệp ngày càng phụ thuộc vào nền tảng điện toán đám mây (Cloud Computing). Việc triển khai hệ thống trên Cloud giúp tăng khả năng mở rộng, tối ưu chi phí vận hành và cải thiện tốc độ triển khai dịch vụ. Tuy nhiên, thay vì chỉ sử dụng một nhà cung cấp duy nhất, nhiều tổ chức lựa chọn mô hình Hybrid Cloud hoặc Multi-Cloud để tận dụng ưu điểm của nhiều nền tảng khác nhau, đồng thời tránh phụ thuộc vào một nhà cung cấp.

Khi hệ thống phân tán trên nhiều môi trường (private cloud, public cloud), bài toán quản trị định danh và phân quyền truy cập trở nên phức tạp hơn. Nếu mỗi hệ thống tự quản lý người dùng độc lập, tổ chức sẽ đối mặt với nhiều vấn đề như:

- Người dùng phải đăng nhập nhiều lần trên nhiều hệ thống, trải nghiệm kém.
- Khó kiểm soát chính sách truy cập thống nhất giữa các nền tảng.
- Tăng nguy cơ lộ thông tin xác thực do việc quản lý nhiều tài khoản/mật khẩu.
- Khó theo dõi và kiểm toán tập trung các sự kiện đăng nhập và truy cập tài nguyên.

Do đó, việc xây dựng một cơ chế định danh tập trung, cho phép người dùng đăng nhập một lần và truy cập được nhiều hệ thống, là một nhu cầu thực tế và mang tính cấp thiết trong vận hành Cloud hiện đại.

1.2. Lý do chọn đề tài

Trong môi trường Multi-Cloud, hệ thống thường bao gồm nhiều dịch vụ/ứng dụng khác nhau, mỗi dịch vụ lại có cơ chế xác thực và phân quyền riêng. Vấn đề đặt ra là:

- Làm sao để hợp nhất trải nghiệm đăng nhập?
Người dùng cần một cơ chế “đăng nhập một lần” nhưng vẫn đảm bảo truy cập đúng tài nguyên được phép.
- Làm sao để đảm bảo an toàn truy cập xuyên hệ thống?
Nếu triển khai sai hoặc thiếu kiểm soát, cơ chế đăng nhập tập trung có thể trở thành điểm yếu nghiêm trọng vì chỉ cần chiếm quyền tài khoản định danh là có thể mở rộng tấn công sang nhiều hệ thống.
- Làm sao để giám sát và phát hiện bất thường?
Trong hệ thống phân tán, nếu không có logging và monitoring hợp lý, việc phát hiện truy cập đáng ngờ hoặc điều tra sự cố sẽ rất khó khăn.

Như vậy, bài toán không chỉ là “SSO hoạt động được”, mà cần hướng đến “SSO hoạt động ổn định, an toàn và có khả năng giám sát/kiểm soát”.

1.3. Mục tiêu đề tài

Đề tài hướng đến việc thiết kế và triển khai cơ chế quản trị định danh và đăng nhập tập trung trong môi trường Cloud. Các mục tiêu chính gồm:

1.3.1. Mục tiêu kỹ thuật

- Xây dựng mô hình xác thực tập trung (Identity Provider – IdP) cho nhiều hệ thống (Service Provider – SP).
- Triển khai cơ chế đăng nhập một lần (Single Sign-On – SSO) để người dùng truy cập xuyên nhiều nền tảng Cloud.
- Thiết lập cơ chế liên kết định danh (Federation) giữa các môi trường khác nhau.

1.3.2. Mục tiêu bảo mật và vận hành

- Đảm bảo phân quyền truy cập theo nguyên tắc Least Privilege.
- Áp dụng các cơ chế tăng cường bảo mật như giới hạn thời gian phiên đăng nhập, kiểm soát token, MFA (nếu có).

- Thu thập và phân tích log xác thực nhằm hỗ trợ giám sát và kiểm toán.
- Đề xuất phương án xử lý khi xảy ra sự cố liên quan đến định danh (tài khoản bị đánh cắp, truy cập bất thường...).

1.4. Phạm vi đề tài

1.4.1. Đối tượng nghiên cứu

- Các thành phần trong hệ thống xác thực và phân quyền: Identity Provider (IdP), Service Provider (SP), tài khoản người dùng, token và session.
- Các mô hình triển khai định danh tập trung trong môi trường Cloud.
- Các giao thức xác thực phổ biến dùng trong SSO và Federation.

1.4.2. Phạm vi đề tài

- Đề tài tập trung vào môi trường Cloud mang tính mô phỏng/lab, triển khai trên nhiều nền tảng khác nhau.
- Xem xét hai nhóm giao thức chính thường dùng trong SSO:
- Nhóm dựa trên SAML
- Nhóm dựa trên OIDC/OAuth
- Nghiên cứu theo hướng triển khai + đánh giá thực nghiệm và mô phỏng các tình huống rủi ro thường gặp.

1.5. Phương pháp thực hiện

Đề tài được thực hiện theo các bước:

- Nghiên cứu lý thuyết: Cloud IAM, SSO, Federation, các giao thức xác thực.
- Phân tích bài toán thực tế: xác định yêu cầu đăng nhập và phân quyền khi triển khai đa nền tảng.
- Thiết kế kiến trúc hệ thống: xác định vai trò IdP/SP, luồng đăng nhập và mapping quyền.
- Triển khai thực nghiệm: cấu hình hệ thống định danh, tích hợp với các dịch vụ Cloud.

- Đánh giá và kiểm thử: kiểm tra hoạt động đăng nhập, phân quyền và mô phỏng một số tình huống rủi ro.
- Tổng hợp kết quả và đề xuất cải tiến.

1.6. Ý nghĩa khoa học và thực tiễn

1.6.1. Ý nghĩa thực tiễn

- Giảm số lượng tài khoản phân tán, tăng tính tập trung trong quản trị người dùng.
- Tăng trải nghiệm người dùng khi truy cập nhiều hệ thống Cloud.
- Hỗ trợ vận hành hệ thống Cloud theo hướng quản trị dễ hơn và an toàn hơn.

1.6.2. Ý nghĩa kỹ thuật

- Cung cấp một mô hình tham khảo để triển khai SSO/Federation.
- Minh họa cơ chế hoạt động của các giao thức xác thực hiện đại.
- Đưa ra đánh giá và bài học kinh nghiệm về rủi ro cấu hình sai trong Identity System.

Chương 2. CƠ SỞ LÝ THUYẾT VÀ NỀN TẢNG

2.1. Tổng quan về định danh số (Digital Identity) và IAM

2.1.1. Khái niệm định danh số

Định danh số (Digital Identity) là tập hợp các thông tin mô tả một thực thể trong hệ thống (con người hoặc máy), có thể bao gồm:

- Thông tin định danh: username, email, UUID, tenant/domain
- Thuộc tính (attributes): phòng ban, lớp, nhóm quyền, trạng thái tài khoản
- Thông tin xác thực (credentials): mật khẩu, OTP secret, khóa riêng, chứng chỉ số...

Trong hệ thống Cloud, định danh số không chỉ dành cho “người dùng” mà còn gồm:

- Service account / machine identity (ứng dụng, agent)
- Role / group identity (vai trò ảo đại diện quyền)
- Workload identity (pod/container, function)

2.1.2. IAM là gì?

Identity and Access Management (IAM) là tập hợp chính sách + công nghệ giúp quản lý:

- Identity Lifecycle: tạo, sửa, khóa, xóa tài khoản
- Authentication: xác thực ai đang truy cập
- Authorization: cấp quyền truy cập tài nguyên
- Auditing: ghi nhận hoạt động để kiểm toán

IAM trong Cloud là “xương sống” của bảo mật vì mọi truy cập đến tài nguyên đều phải đi qua cơ chế IAM (API, console, dashboard, CLI...).

2.2. Xác thực và phân quyền trong hệ thống Cloud

2.2.1. Xác thực (Authentication)

Authentication trả lời câu hỏi “Bạn là ai?”. Các phương thức phổ biến:

- Knowledge Factor – thứ người dùng biết
 - Password, PIN
 - Câu hỏi bảo mật (không khuyến nghị)
- Possession Factor – thứ người dùng có
 - OTP (Google Authenticator, Microsoft Authenticator)
 - Hardware token, smart card
 - SSH key, certificate
- Inherence Factor – thứ thuộc về người dùng
 - Sinh trắc học: vân tay, khuôn mặt
 - Trong thực tế, Cloud khuyến nghị xác thực theo MFA (ít nhất 2 yếu tố) cho tài khoản quan trọng.

2.2.2. Phân quyền (Authorization)

Authorization trả lời câu hỏi “Bạn được làm gì?”. Trong Cloud thường dùng các mô hình:

a) RBAC (Role-Based Access Control)

Quyền được gán theo vai trò (role).

Ví dụ:

- admin: full access
- developer: deploy + read logs
- viewer: chỉ xem

Ưu điểm: dễ hiểu, dễ quản trị.

Nhược điểm: nếu role thiết kế kém dễ bị “role phình to”, cấp quyền dư.

b) ABAC (Attribute-Based Access Control)

Quyền dựa trên thuộc tính và chính sách.

Ví dụ: Cho phép nếu department=IT và resource=prod và time<18:00

Ưu điểm: linh hoạt, kiểm soát theo ngữ cảnh tốt.

Nhược điểm: khó triển khai, khó debug policy.

c) Nguyên tắc Least Privilege & Separation of Duties

Least Privilege: chỉ cấp đúng quyền cần thiết

Separation of Duties: tách vai trò vận hành (operator) và vai trò quản trị (admin) để giảm rủi ro lạm quyền

2.3. Bài toán định danh trong Multi-Cloud

2.3.1. Credential Sprawl (bùng nổ thông tin đăng nhập)

Nếu mỗi hệ thống dùng user riêng → người dùng có nhiều tài khoản và mật khẩu → tăng nguy cơ:

- đặt mật khẩu yếu / dùng lại mật khẩu
- tài khoản “ma” tồn tại lâu nhưng không dùng
- khó thu hồi đồng loạt khi nhân sự nghỉ việc

2.3.2. Chính sách truy cập không đồng nhất

Các nền tảng Cloud có cơ chế permission/role khác nhau. Nếu không quản trị thống nhất sẽ dẫn đến:

- không đồng bộ nhóm quyền giữa các hệ thống
- sai mapping dẫn đến cấp quyền quá mức

2.3.3. Khó giám sát và điều tra sự cố

Log phân tán nhiều nguồn → khó trả lời:

- Ai đăng nhập khi nào?

- Từ đâu truy cập?
- Sau đăng nhập đã làm gì?
- Có bất thường không?
- Do đó cần có cơ chế “hợp nhất định danh” + “giám sát tập trung”.

2.4. Single Sign-On (SSO) và lợi ích trong hệ thống Cloud

2.4.1. Khái niệm SSO

Single Sign-On (SSO) là cơ chế cho phép người dùng đăng nhập 1 lần và sử dụng phiên đăng nhập đó để truy cập nhiều dịch vụ mà không cần nhập lại mật khẩu.

Ví dụ thực tế: Đăng nhập vào tài khoản tổ chức → truy cập được dashboard Cloud, tool monitoring, repo code, mail nội bộ...

2.4.2. Lợi ích của SSO

Đối với người dùng

- thuận tiện, giảm thao tác đăng nhập
- giảm lỗi quên mật khẩu

Đối với tổ chức

- tập trung chính sách bảo mật (MFA, password policy)
- dễ quản lý lifecycle tài khoản
- dễ kiểm toán hơn (một điểm kiểm soát)

2.4.3. Rủi ro của SSO

SSO là “cổng chính”, nên nếu bị compromise có thể dẫn đến:

- chiếm quyền truy cập nhiều hệ thống cùng lúc
- tấn công lan rộng (lateral movement)

Vì vậy SSO phải đi kèm: MFA, giới hạn session, logging, rule phát hiện bất thường.

2.4.4. Các giao thức triển khai SSO phổ biến

2.4.4.1. SAML 2.0

a) Tổng quan

SAML 2.0 (Security Assertion Markup Language) là chuẩn phổ biến trong doanh nghiệp để triển khai SSO qua trình duyệt.

- Assertion thường ở dạng XML
- SSO dựa trên redirect giữa SP và IdP
- Dùng XML Signature để đảm bảo tính toàn vẹn

b) Các thành phần quan trọng trong SAML

- Assertion: gói dữ liệu định danh, quyền, thời gian hiệu lực
- Issuer: ai phát hành assertion (IdP)
- Subject: người dùng
- Conditions: điều kiện hiệu lực (NotBefore/NotOnOrAfter)
- Audience Restriction: assertion chỉ dùng cho SP cụ thể
- Signature: chữ ký số

c) Các luồng SAML thường gặp

SP-initiated flow

- User truy cập SP
- SP redirect đến IdP kèm SAMLRequest
- User login tại IdP
- IdP trả SAMLResponse về SP (qua POST)
- SP verify → tạo session

IdP-initiated flow

- User login trước tại IdP
- IdP redirect đến SP kèm SAMLResponse
- SP verify → tạo session

d) Ưu/nhược điểm SAML

Ưu điểm

- Chuẩn enterprise lâu đời, nhiều hệ thống hỗ trợ
- Phù hợp với portal/console nội bộ

Nhược điểm

- XML phức tạp, cấu hình dễ sai
- Debug khó hơn OIDC/JWT

2.4.4.2. OAuth2

a) Tổng quan

OAuth2 là framework dùng để ủy quyền truy cập mà không cung cấp mật khẩu cho bên thứ ba.

Token chính:

- Access Token: dùng gọi API
- Refresh Token: làm mới phiên truy cập

b) Các grant type phổ biến

- Authorization Code (web app)
- Client Credentials (machine-to-machine)
- Device Code (thiết bị không có browser)
- Refresh Token grant
- OAuth2 chủ yếu phục vụ “phân quyền”, còn định danh sẽ do OIDC mở rộng.

2.4.4.3. OpenID Connect (OIDC)

a) Tổng quan

- OIDC = OAuth2 + lớp xác thực định danh.
- OIDC cung cấp:
- ID Token (JWT): chứa danh tính user
- Access Token: truy cập API
- UserInfo endpoint: trả thông tin user

b) ID Token chứa gì?

ID Token thường là JWT gồm:

- iss (issuer): ai phát hành
- sub (subject): user ID
- aud (audience): token dùng cho client nào
- exp, iat (time)
- claim mở rộng: email, name, groups...

c) Luồng phổ biến: Authorization Code Flow

- Client redirect user đến IdP
- User login và consent
- Client nhận authorization code
- Client đổi code lấy token
- Client verify ID token → login thành công

d) Ưu/nhược điểm OIDC

Ưu điểm

- JWT nhẹ, dễ verify signature
- Tối ưu cho hệ thống hiện đại, web/API

Nhược điểm

- Nếu token bị lộ → bị lạm dụng
- Refresh token quản trị kém → long-lived compromise

2.4.4.4. So sánh tổng quát SAML và OIDC

Tiêu chí	SAML 2.0	OIDC
Dạng dữ liệu	XML Assertion	JWT Token
Mục tiêu chính	SSO web/enterprise	Identity cho web/app/API
Debug và tích hợp	Khó hơn	Dễ hơn
Rủi ro chính	Replay, XML config sai	Token theft, refresh abuse
Phổ biến trong	Portal, console, enterprise app	Modern app, microservice

2.5. Federation (liên kết định danh) trong kiến trúc hiện đại

2.5.1. Khái niệm Federation

Identity Federation là mô hình trong đó một hệ thống (Service Provider – SP) tin tưởng kết quả xác thực từ một hệ thống khác (Identity Provider – IdP), thay vì tự lưu user/pass.

Thành phần:

- Identity Provider (IdP): nơi xác thực người dùng
- Service Provider (SP): nơi cung cấp dịch vụ
- Trust relationship: khóa ký, certificate, metadata, endpoint...

2.5.2. SSO và Federation khác nhau thế nào?

Federation: nói về mối quan hệ tin cậy giữa nhiều domain/hệ thống

SSO: nói về trải nghiệm đăng nhập một lần

Thực tế: đa số hệ thống SSO doanh nghiệp đều dựa trên Federation.

2.6. Ánh xạ định danh và phân quyền (Identity Mapping & Role Mapping)

2.6.1. Mapping danh tính

Trong Federation, hệ thống thường cần ánh xạ:

- user trong IdP $\leftrightarrow$ user/subject trong SP
- nhóm (group) $\leftrightarrow$ role quyền

Ví dụ:

- group=cloud-admin $\rightarrow$ role admin
- group=devops $\rightarrow$ role deploy, read logs

2.6.2. Mapping quyền: nguyên tắc và sai lầm thường gặp

Nguyên tắc

- role mapping theo nhóm thay vì gán trực tiếp từng user
- mặc định “deny” nếu không có mapping
- hạn chế quyền đặc biệt (admin) chỉ cho nhóm nhỏ

Sai lầm thường gặp

- map nhầm nhóm $\rightarrow$ cấp quyền vượt mức
- dùng role “*” hoặc policy quá rộng
- không kiểm soát attribute claim (nhận group từ token mà không validate nguồn)

2.7. Các vấn đề bảo mật thường gặp trong SSO/Federation

2.7.1. Replay Attack

Kẻ tấn công sử dụng lại assertion/token hợp lệ trong thời gian còn hiệu lực.

Giảm thiểu

- token TTL ngắn
- kiểm tra nonce, session binding, one-time use (tùy hệ thống)

2.7.2. Token Theft / Session Hijacking

Token bị đánh cắp qua:

- lộ log
- lộ cookie
- XSS trong ứng dụng
- MITM nếu thiếu TLS

Giảm thiểu

- TLS bắt buộc
- HttpOnly/Secure cookie
- token TTL ngắn
- hạn chế lưu token ở client

2.7.3. Signature Bypass / Unsigned Token

Nếu SP không kiểm tra signature đầy đủ → attacker giả token.

Giảm thiểu

- luôn verify chữ ký + issuer + audience
- không chấp nhận thuật toán “none”
- pinning metadata/cert

2.7.4. Misconfiguration

Các lỗi hay gặp:

- sai redirect URI
- sai audience/issuer
- ACS endpoint mapping sai
- clock skew cấu hình thiếu chặt chẽ

2.7.5. Privilege Escalation

Một user có quyền thấp nhưng “leo thang” nhờ:

- mapping role sai
- policy quá rộng
- group claim bị giả mạo

2.8. Hardening cho hệ thống định danh tập trung

2.8.1. Hardening ở Identity Provider (IdP)

- Bật MFA cho nhóm quan trọng
- Áp dụng policy mật khẩu mạnh (hoặc passwordless)
- Giới hạn session duration
- Khóa account sau nhiều lần login fail
- Theo dõi admin events, hạn chế quyền admin

2.8.2. Hardening ở Service Provider (SP)

- Chỉ tin token/assertion từ IdP xác định
- Validate issuer, signature, audience, expiry
- Chặn redirect không hợp lệ
- Không cấp quyền mặc định nếu claim thiếu

2.8.3. Hardening token và khóa ký

- Access token TTL ngắn
- Refresh token rotation (nếu hỗ trợ)
- định kỳ rotate certificate / signing key
- không ghi token nhạy cảm vào log

2.9. Logging – Monitoring – Auditing trong hệ thống Cloud

2.9.1. Vì sao cần giám sát?

Giám sát giúp:

- phát hiện truy cập bất thường sớm
- giảm thời gian phát hiện sự cố (MTTD)

- hỗ trợ điều tra sau sự cố (forensics)

2.9.2. Log cần thu thập

Có thể chia 3 nhóm:

Nhóm 1: Xác thực

- login success/failure
- MFA success/failure
- token issued/refresh/revoke

Nhóm 2: Quản trị

- tạo user, reset password
- thay đổi group/role mapping
- thay đổi config client/realm

Nhóm 3: Truy cập tài nguyên

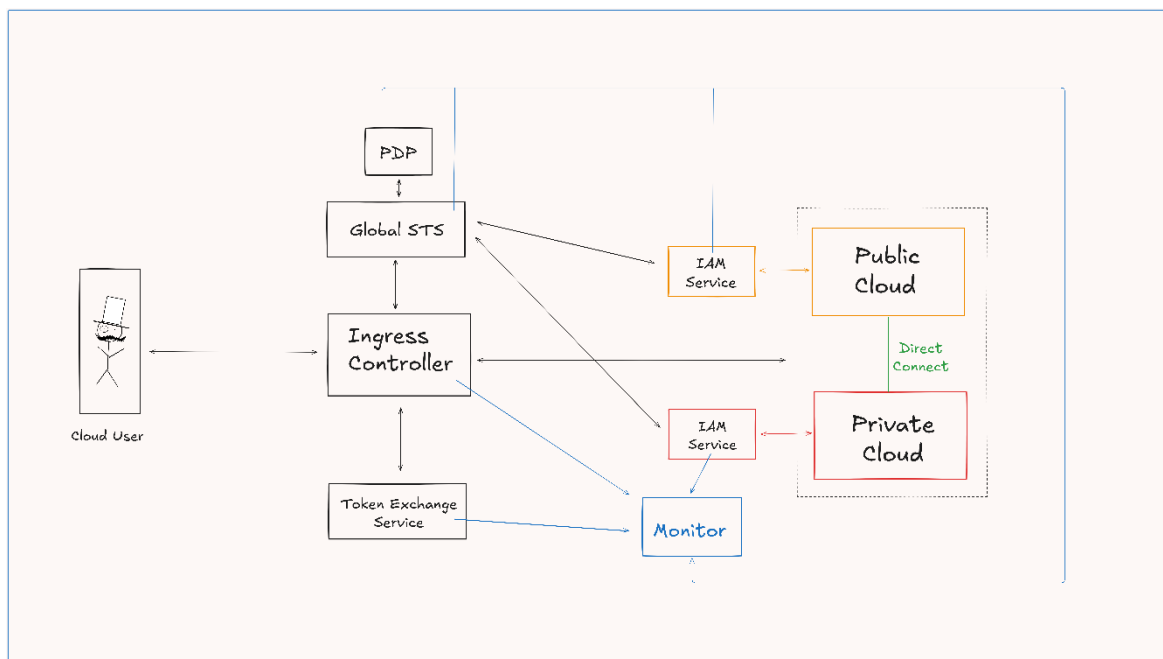
- hành động trên dashboard/API
- assume role / access denied
- thay đổi cấu hình cloud resource

2.9.3. Chỉ số và dấu hiệu bất thường (Indicators)

- Nhiều lần đăng nhập thất bại liên tục (bruteforce)
- Đăng nhập từ IP/địa lý bất thường
- Đăng nhập thành công sau chuỗi thất bại
- Một tài khoản đăng nhập đồng thời nhiều nơi
- Token refresh bất thường với tần suất cao
- Thao tác admin xảy ra ngoài giờ

Chương 3. KIẾN TRÚC & TRIỂN KHAI THỰC TẾ

3.1. Kiến trúc giải pháp

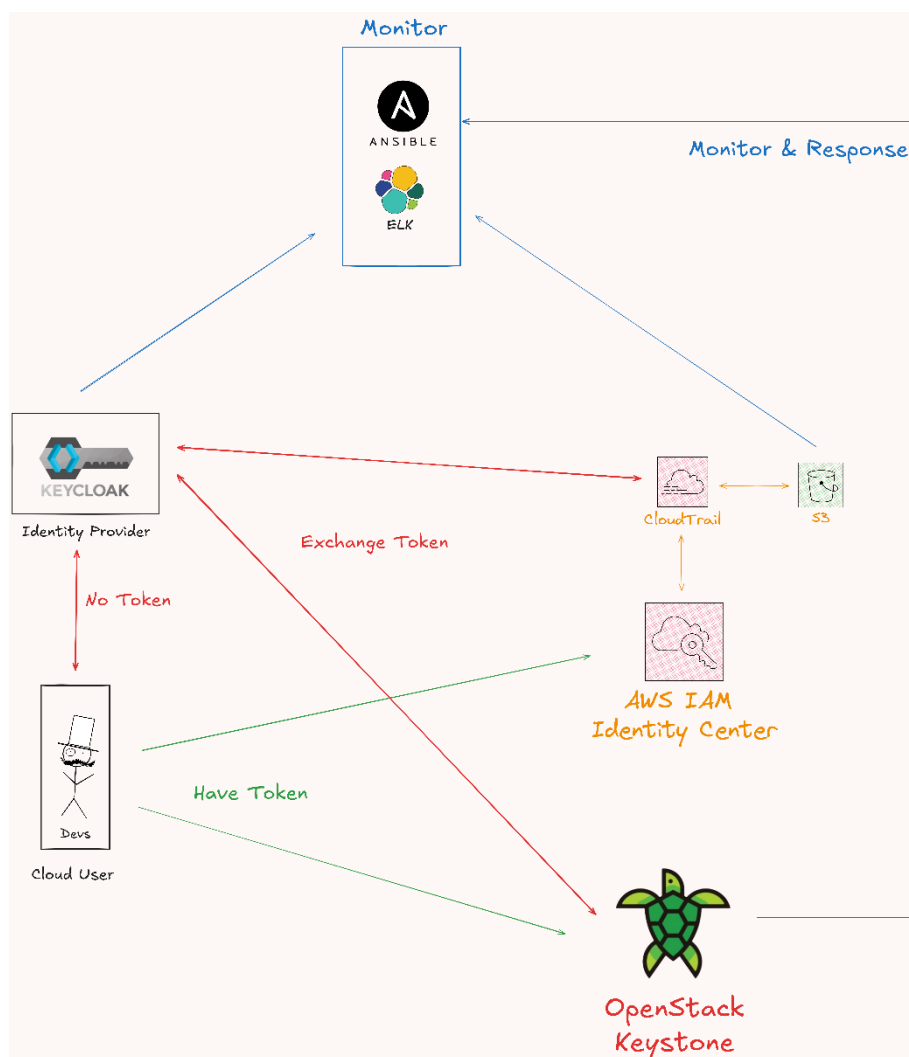


Kịch bản: Hệ thống được thiết kế cho doanh nghiệp vận hành trên mô hình Hybrid Cloud, nhằm quản lý IAM tập trung cho nhân viên, hợp nhất các cơ chế IAM đa nền tảng, áp dụng chính sách truy cập thống nhất, đồng thời đảm bảo truy cập an toàn và linh hoạt thông qua cơ chế cấp phát token tạm thời.

- Global STS (Security Token Service): Đóng vai trò là lớp Identity Fabric bao trùm lên hệ thống, hợp nhất các hệ thống IAM của từng platform.
- Ingress Controller: Là Single Entry Point cho người dùng hạ tầng cloud. Phối hợp với Global STS để xác thực danh tính ban đầu, đảm bảo các yêu cầu chỉ được chuyển tiếp khi đã được xác thực và cấp token hợp lệ.
- Token Exchange Service: Chuyển Home Token (định danh trong nội bộ công ty) thành Target Token của nền tảng cloud tương ứng.
- PDP (Policy Decision Point): Là nơi quản lý và ra quyết định chính sách truy cập tập trung, đánh giá dựa vào danh tính/ngữ cảnh và trả về quyết định Allow/Deny cho Global STS.

- IAM Service (Public Cloud / Private Cloud): Nhận token đã được chuyển đổi từ Global STS, ánh xạ sang quyền truy cập cụ thể trên từng nền tảng.
- Monitor: Đóng vai trò giám sát và ghi nhận hoạt động toàn hệ thống

3.2. Kiến trúc triển khai



Vì thời gian có hạn nên nhóm triển khai đơn giản hơn:

- Keycloak đóng vai trò là Global STS, Token Exchange Service và cả PDP
- User chỉ cần xác thực ở Keycloak, Keycloak Token đóng vai trò là Home Token.
- Tất cả được giám sát ở node ELK. Response bằng Ansible Playbook.

3.3. Triển khai Identity Provider & Service Provider

Keycloak đóng vai trò là Identity Provider (IdP) trung tâm, cung cấp khả năng xác thực một lần (SSO) cho cả hai nền tảng OpenStack và AWS thông qua các giao thức OIDC và SAML 2.0.

3.3.1. Cài đặt và thiết lập môi trường

Hệ thống được triển khai trên nền tảng Ubuntu Server với các thành phần cốt lõi:

- Java Runtime: Sử dụng OpenJDK 21 để đảm bảo hiệu suất và tương thích với Keycloak phiên bản mới nhất (26.x).
- Thiết lập Keycloak chạy dưới dạng một Docker Container nằm ngoài hạ tầng Hybrid Cloud:
 - Cổng dịch vụ: 8088 (để tránh xung đột với các dịch vụ hệ thống khác).
 - Cấu hình --hostname-strict=false để linh hoạt trong việc truy cập qua IP hoặc Domain trong môi trường Lab.

3.3.2. Cấu hình Realm và Federation cho OpenStack (OIDC)

Để tích hợp với OpenStack Keystone, Keycloak được cấu hình như một OIDC Provider:

- Tạo Realm: Khởi tạo Realm tên là Cloud để tách biệt quản lý với hệ thống mặc định.
- Cấu hình Client keystone: protocol openid-connect.
- Access Type: confidential (yêu cầu Client Secret để tăng cường bảo mật).
- Redirect URIs: Cấu hình các endpoint của Keystone như .../v3/auth/OS-FEDERATION/identity_providers/keycloak/protocols/openid/websso để tiếp nhận token sau khi xác thực thành công.
- Identity Mapping: Thiết lập cơ chế ánh xạ người dùng từ Keycloak sang Keystone dựa trên các thuộc tính preferred_username và email để đảm bảo định danh người dùng đồng nhất trên môi trường Private Cloud.

3.3.3. Cấu hình Federation cho AWS Identity Center (SAML)

Đối với AWS, Keycloak đóng vai trò là SAML IdP. Quá trình triển khai thực hiện qua việc trao đổi Metadata giữa hai hệ thống:

- Trao đổi Metadata: Xuất file metadata từ AWS IAM Identity Center và import vào Keycloak để tự động cấu hình các endpoint SAML.
- Cấu hình Client Scopes: Tạo scope aws và thiết lập Mapper để định dạng NameID theo địa chỉ email (urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress). Đây là yêu cầu bắt buộc để AWS có thể định danh người dùng từ hệ thống bên ngoài.
- Trust Relationship: Upload Keycloak SAML Metadata ngược lại lên AWS để hoàn tất chuỗi tin cậy (Chain of Trust) giữa IdP và SP.

3.4. Triển khai ELK Pipeline với TLS

3.4.1. Thiết lập hạ tầng ELK

ELK Stack được triển khai tập trung (All-in-One) trên một VM nằm ngoài hạ tầng Hybrid Cloud, bao gồm:

- Elasticsearch: Đóng vai trò cơ sở dữ liệu phân tán, lưu trữ toàn bộ dữ liệu nhật ký.
- Kibana: Giao diện quản trị và trực quan hóa dữ liệu, giúp truy vấn và tạo các Dashboard an ninh.
- Logstash: Bộ máy xử lý dữ liệu (ETL), tiếp nhận nhật ký từ nhiều nguồn, lọc và chuyển đổi định dạng trước khi đẩy vào Elasticsearch.

3.4.2. Hardening

Hệ thống được cấu hình bảo mật đa lớp thông qua giao thức TLS và cơ chế xác thực RBAC.

3.4.2.1. Certificate Management

Hệ thống sử dụng công cụ `elasticsearch-certutil` để khởi tạo một hạ tầng khóa công khai (PKI) nội bộ:

- Certificate Authority (CA): Một CA tự ký được tạo ra để làm gốc tin cậy cho toàn bộ các thành phần trong Stack.
- Chứng chỉ định danh (Node Certificates): Các chứng chỉ được cấp phát riêng cho Elasticsearch, Kibana và Logstash dựa trên tệp cấu hình `instance.yml`. Việc ràng buộc chứng chỉ với địa chỉ IP và DNS giúp ngăn chặn các cuộc tấn công giả mạo node (Node Impersonation).

3.4.2.2. Encryption in Transit

Hệ thống kích hoạt tính năng X-Pack Security và cấu hình TLS trên hai tầng giao tiếp cốt lõi:

- Transport Layer SSL: Mã hóa toàn bộ lưu lượng trao đổi nội bộ giữa các thành phần của ELK.
- HTTP Layer SSL: Sử dụng giao thức HTTPS (cổng 9200 và 5601) để bảo vệ dữ liệu khi người dùng truy cập Dashboard hoặc khi các Agent (Filebeat) đẩy log về.
- Riêng với Logstash, Private Key được chuyển đổi sang định dạng PKCS#8 để đảm bảo khả năng tương thích.

3.4.2.3. Authentication & RBAC

Thay vì sử dụng cấu hình mặc định không mật khẩu, hệ thống được thắt chặt bằng cách:

- Vô hiệu hóa truy cập ẩn danh: Kích hoạt `xpack.security.enabled: true` bắt buộc mọi yêu cầu API phải có thông tin xác thực.
- Quản lý người dùng tích hợp: Thiết lập mật khẩu mạnh cho các tài khoản hệ thống như `elastic`, `kibana` và `logstash_system`.

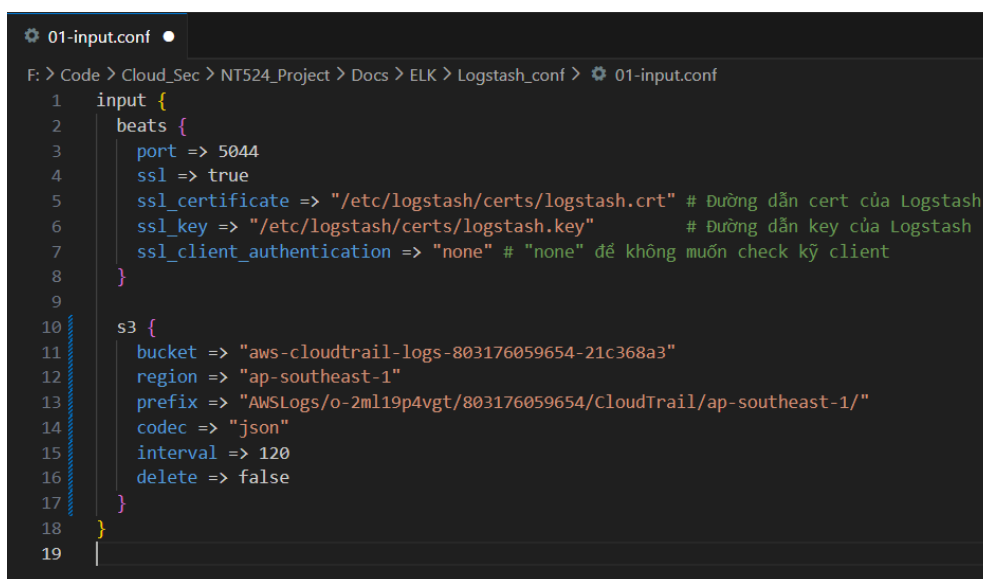
- Nguyên tắc Least Privilege: Logstash được cấp một tài khoản riêng với vai trò giới hạn (Role-based) chỉ đủ để thực hiện ghi dữ liệu vào các Index cụ thể, giảm thiểu rủi ro nếu Agent bị chiếm quyền điều khiển.

3.5. Logstash ETL Pipeline

3.5.1. Input Layer

Logstash được cấu hình để lắng nghe đồng thời từ hai nguồn dữ liệu:

- Beats Input (Port 5044): Tiếp nhận dữ liệu thời gian thực từ Filebeat cài đặt trên các node Keycloak và OpenStack Keystone. Đường truyền này được bảo mật bằng TLS để đảm bảo tính bảo mật & toàn vẹn của log.
- S3 Input (AWS CloudTrail): Logstash đóng vai trò là một client chủ động kết nối tới S3 Bucket của AWS theo chu kỳ 120 giây để lấy các file log CloudTrail (.json.gz).



```
01-input.conf
F: > Code > Cloud_Sec > NT524_Project > Docs > ELK > Logstash_conf > 01-input.conf
1  input {
2    beats {
3      port => 5044
4      ssl => true
5      ssl_certificate => "/etc/logstash/certs/logstash.crt" # Đường dẫn cert của Logstash
6      ssl_key => "/etc/logstash/certs/logstash.key" # Đường dẫn key của Logstash
7      ssl_client_authentication => "none" # "none" để không muốn check kỹ client
8    }
9  }
10
11  s3 {
12    bucket => "aws-cloudtrail-logs-803176059654-21c368a3"
13    region => "ap-southeast-1"
14    prefix => "AWSLogs/o-2ml19p4vgt/803176059654/CloudTrail/ap-southeast-1/"
15    codec => "json"
16    interval => 120
17    delete => false
18  }
19 }
```

Cấu hình Logstash input

3.5.2. Process Layer

Do mỗi nền tảng có định dạng log khác nhau, hệ thống sử dụng các bộ lọc chuyên biệt để trích xuất thông tin nghiệp vụ:

- Xử lý nhật ký Keycloak: Sử dụng plugin grok để tách tiêu đề log và plugin kv (Key-Value) để tự động bóc tách các tham số sự kiện như auth_method, realmName, và client_id. Các log không quan trọng từ Hibernate hoặc SQL được loại bỏ để tối ưu hóa tài nguyên.
- Xử lý nhật ký OpenStack Keystone: Hệ thống phân loại log thành hai dạng: Apache Access logs (để theo dõi luồng Redirect/Callback OIDC) và Python App logs (để theo dõi quá trình ánh xạ người dùng và gán Role RBAC). Các sự kiện quan trọng như tạo Token thành công (token_created) hoặc thất bại được gắn nhãn cụ thể, còn lại sẽ bị lọc bớt.
- Xử lý nhật ký AWS CloudTrail: Sử dụng plugin split để tách mảng Records trong file log S3 thành từng sự kiện riêng lẻ. Hệ thống tập trung lọc các sự kiện UserAuthentication và Authenticate.

[Các file .conf em gửi kèm báo cáo.](#)

3.5.3. Enrichment & Output

Sau khi bóc tách, mọi bản ghi đều trải qua các bước xử lý chung:

- GeoIP Lookup: Tự động tra cứu địa chỉ IP của người dùng để xác định vị trí địa lý (Thành phố, Quốc gia), phục vụ việc phát hiện các cảnh báo.
- Chuẩn hóa Schema: Mọi log từ AWS, OpenStack hay Keycloak đều được map về các trường chung: timestamp, user_id, ip, geometry, event_type, idp (Identity Provider), user_agent, sp (Service Provider), và outcome (success/failure).
- Đầu ra an toàn: Dữ liệu sau xử lý được đẩy vào Elasticsearch qua giao thức HTTPS, lưu trữ theo các Index phân chia theo ngày (sso-logs-yyyy.MM.dd) để dễ dàng quản lý.

Ánh xạ dữ liệu từ các nguồn về Common Schema:

Schema	Nguồn Keycloak (IdP)	Nguồn OpenStack (SP)	Nguồn AWS (SP)	Ý nghĩa
user_id	username hoặc userId	token (Apache) hoặc temp_os_username (Python)	UserName (Sign-in) hoặc userId (SSO)	Định danh duy nhất của người dùng thực hiện hành động.
event_type	type (LOGIN, CODE_T O_TOK EN,...)	Phân loại dựa trên request_path (sso_initiate, token_created,.. .)	eventName (UserAuthen tication, Authenticat e)	Loại hành động/sự kiện diễn ra.
outcome	Phân loại dựa trên _ERROR hoặc level	Dựa trên response_code (2xx/3xx là success)	Dựa trên errorCode hoặc UserAuthentic ation status	Kết quả của hành động (Thành công/Thất bại).
ip	ipAddress	ip (từ Grok Apache/Python)	sourceIPAddre ss	Địa chỉ IP nguồn thực hiện yêu cầu.
client_id	clientId (ví dụ: keystone, aws-iam- idp)	project_id (trích xuất từ RBAC log)	recipientAcco untId (ID tài khoản AWS)	Ứng dụng hoặc môi trường đích nhận yêu cầu.
idp	Mặc định: Keycloak	Mặc định: Keycloak	Mặc định: Keycloak	Identity Provider
sp	Mặc định: Keycloak	Mặc định: OpenStack_Keysto ne	Mặc định: AWS_IAM_Id entity_Center	Service Provider
attributes	Tổng hợp auth_metho d, realmName	Tổng hợp Referrer, Method, Roles, Project	Tổng hợp LoginTo, Method, IdentityStore	Chứa các thông tin bổ sung đặc thù của từng nguồn log.

3.6. Kibana Detection Rule: Impossible Travel

Quy tắc "Impossible Travel" được thiết lập để phát hiện dấu hiệu chiếm đoạt tài khoản, khi có nhiều lần đăng nhập thành công từ các vị trí địa lý cách xa nhau, trong một khoảng thời gian ngắn mà con người không thể di chuyển kịp.

3.6.1. Xây dựng dữ liệu lịch sử (Transform)

- Hệ thống sử dụng tính năng Kibana Transform để tổng hợp dữ liệu từ Data View sso-logs-.*.
- Dữ liệu được nhóm theo user_id để theo dõi vị trí đăng nhập cuối cùng (max @timestamp) và tọa độ tương ứng (top_metrics.location).

3.6.2. Làm giàu dữ liệu thời gian thực (Enrich Policy)

Một Enrich Policy tên là user-location-policy được thiết lập để kết nối nhật ký đăng nhập mới với dữ liệu vị trí gần nhất trong quá khứ. Khi một người dùng đăng nhập, hệ thống sẽ ngay lập tức đối chiếu tọa độ hiện tại với tọa độ lưu trong lịch sử.

3.6.3. Tính toán vận tốc di chuyển (Scripted Field)

Sử dụng ngôn ngữ Painless để tính toán khoảng cách giữa hai điểm dựa trên công thức Haversine:

$$d = 2R \cdot \arctan 2(\sqrt{a}, \sqrt{1-a})$$

Trong đó:

$$a = \sin^2\left(\frac{\Delta lat}{2}\right) + \cos(lat_1) \cdot \cos(lat_2) \cdot \sin^2\left(\frac{\Delta lon}{2}\right)$$

Ngưỡng cảnh báo: Hệ thống tính toán trường *travel_velocity* (vận tốc di chuyển).

Nếu vận tốc này vượt quá ngưỡng **hợp lý** (ví dụ: di chuyển hơn 5,000 km trong vòng 5 phút), một cảnh báo (Alert) sẽ được kích hoạt ngay lập tức.

3.7. Ansible Playbook: Revoke Session

```
F: > Code > Cloud_Sec > NT524_Project > IR > Playbooks > ! revoke_session.yml > ...
1  |---
2  - name: Incident Response - Revoke Token
3    hosts: localhost
4    gather_facts: no
5    vars:
6      # Cấu hình Keycloak
7      keycloak_url: "https://sso.hippi.work"
8      realm_name: "cloud" # Realm chứa user (ví dụ: master hoặc customer)
9      admin_user: "{{ lookup('env', 'KEYCLOAK_ADMIN_USER') }}" # User có quyền quản trị Keycloak
10     admin_pass: "{{ lookup('env', 'KEYCLOAK_ADMIN_PASSWORD') }}"
11
12   tasks:
13     # BƯỚC 1: Lấy Admin Access Token để được quyền gọi API
14     - name: 1. Authenticate to Keycloak
15       uri:
16         url: "{{ keycloak_url }}/realms/master/protocol/openid-connect/token"
17         method: POST
18         body_format: form-urlencoded
19         body:
20           username: "{{ admin_user }}"
21           password: "{{ admin_pass }}"
22           grant_type: "password"
23           client_id: "admin-cli"
24         validate_certs: no
25       register: keycloak_auth
26       failed_when: keycloak_auth.status != 200
27
28     # BƯỚC 2: Tìm ID (UUID) của user dựa trên username
29     # Keycloak API thao tác qua UUID chứ không phải username
30     - name: 2. Find User ID
31       uri:
32         url: "{{ keycloak_url }}/admin/realms/{{ realm_name }}/users?username={{ suspect_username }}&exact=true"
33         method: GET
34         headers:
35           Authorization: "Bearer {{ keycloak_auth.json.access_token }}"
36         validate_certs: no
37       register: user_info
38
39     - name: Fail if user not found
40       fail:
41         msg: "Không tìm thấy user {{ suspect_username }}"
42       when: user_info.json | length == 0
43
44     # BƯỚC 3: Logout toàn bộ session của User đó (Revoke Consent)
45     - name: 3. Revoke Sessions
46       uri:
47         url: "{{ keycloak_url }}/admin/realms/{{ realm_name }}/users/{{ user_info.json[0].id }}/logout"
48         method: POST
49         headers:
50           Authorization: "Bearer {{ keycloak_auth.json.access_token }}"
51         status_code: 204
52         validate_certs: no
53
54     # BƯỚC 4: Gửi thông báo
55     - name: Notification
56       debug:
57         msg: "Đã thu hồi token của {{ suspect_username }} (UUID: {{ user_info.json[0].id }})"
58
```

Playbook revoke_session.yml

Quy trình thực thi của Playbook *revoke_session.yml*:

- Authentication: Playbook kết nối tới Keycloak Admin API tại để lấy Access Token thông qua tài khoản quản trị admin-cli.

- Find User ID: Do Keycloak quản lý người dùng qua UUID, Ansible sẽ thực hiện truy vấn API để tìm kiếm chính xác user_id (UUID) dựa trên username được gửi từ cảnh báo SIEM. Nếu không tìm thấy người dùng, hệ thống sẽ ghi nhận lỗi và dừng quy trình để tránh nhầm lẫn.
- Revoke Sessions: Sau khi có UUID, Playbook gửi yêu cầu POST tới endpoint /logout của người dùng đó trong Realm tương ứng. Hành động này sẽ ngay lập tức hủy bỏ toàn bộ các Session hiện có của người dùng trên Keycloak.
- Notification: Hệ thống ghi lại nhật ký thực thi, xác nhận việc thu hồi token thành công để phục vụ công tác Forensics sau này.

3.8. Mục tiêu

- Tập trung hóa quản trị định danh (Identity Centralization): Thay vì quản lý tài khoản riêng biệt trên từng Cloud, quy trình giúp thống nhất một nguồn xác thực duy nhất (Single Source of Truth) cho cả OpenStack và AWS thông qua Keycloak, giảm thiểu rủi ro từ việc quản lý quá nhiều tài khoản (Credential Sprawl)
- Minh bạch hóa hoạt động an ninh (Security Visibility): Thu thập và chuẩn hóa nhật ký từ các nguồn khác nhau (On-premise logs, CloudTrail S3) về một giao diện tập trung; biến các dòng nhật ký thô thành thông tin có ý nghĩa (User, IP, Vị trí, Kết quả) thông qua ETL Pipeline.
- Tự động hóa chu trình phản ứng (SOAR - Security Orchestration, Automation and Response): Giúp phát hiện sự cố ngay khi nó xảy ra thay vì đợi quản trị viên kiểm tra log thủ công; việc kết hợp Kibana Alert và Ansible Playbook giúp thực hiện hành động phản ứng tự động, ngăn chặn kẻ tấn công kịp thời thực hiện các hành vi gây hại tiếp theo.
- Đảm bảo tính toàn vẹn và bảo mật hệ thống (Hardening): Đảm bảo toàn bộ quy trình từ xác thực đến truyền tải nhật ký đều được bảo vệ bởi TLS/SSL,

áp dụng nguyên tắc đặc quyền tối thiểu (Least Privilege) nhằm đảm bảo an toàn cho chính hạ tầng giám sát.

Chương 4. KẾT QUẢ VÀ NHẬN XÉT

4.1. Functionality Test

User truy cập Horizon Dashboard, nhập Credential thông qua Keycloak và Login được vào Project Dashboard. Sau đó, truy cập vào AWS SSO Portal, được redirect thẳng tới AWS Management Console (vì token Keycloak vẫn còn hiệu lực)

Tất cả giai đoạn được Log đầy đủ về ELK (Log Keycloak/Keystone là Real-time nhờ Filebeat, log AWS bị delay do cơ chế Pull + cơ chế batch của CloudTrail nhưng vẫn hiện đầy đủ)

Demo:

https://drive.google.com/file/d/1TjOLxGEJ8fo7UI2phOakoOcEAKyJW2Jy/view?usp=drive_link

4.2. MFA Enforcement

Bật tính năng MFA OTP qua Keycloak Web Console cho tài khoản được chọn. Khi đăng nhập lần đầu, Keycloak yêu cầu quét QR bằng Google/Microsoft Authenticator và nhập mã 6 số trong đó. Từ những lần đăng nhập sau, mã OTP 6 số trong Authenticator App đã đăng ký là bắt buộc.

Demo:

https://drive.google.com/file/d/1DsomiBwTJjKQyOZsDoJzArPhzIRQzdve/view?usp=drive_link

4.3. Unsigned Token

OIDC hoạt động dựa trên JWT, mà cấu trúc của JWT là *header.payload.signature*

Thử dùng Burp Suite Repeater bắt OIDC Token và gửi lại tới Keystone:

- Xóa phần signature (Token còn mỗi header.payload): Server báo lỗi *OpenID Connect Provider error: Error in handling response type*.
 - Sửa signature: Server báo lỗi 500 (Internal Error)!
- => **Không thể truy cập với 1 token không có/sai chữ ký!**

Demo: <https://drive.google.com/file/d/1xPBric-IsjhagQGyEuVz8kw95FnPO1OG/view?usp=sharing>

4.4. Alert: Impossible Travel

Đăng nhập lần 1 (bật VPN sang London) qua Keycloak

Mở tab Ẩn danh, đăng nhập lần 2 (Không VPN, vị trí ở Việt Nam)

Rule *Impossible Travel* được trigger, Kibana Alert!

Thực thi playbook *revoke_session* để thu hồi token của tài khoản khả nghi.

Kết quả: Phải login lại vào Keycloak dù chưa hết hạn Token -> Token cũ đã mất hiệu lực!

Demo: https://drive.google.com/file/d/1pWakjtDTkG07PD50c7MPgToRzA8-kF0x/view?usp=drive_link

4.5. Replay Attack

Keycloak bản thân nó có cơ chế chống Replay Attack: *session_code* (ID của phiên) & *execution* (ID của bước trong luồng OIDC)

Kịch bản: Dùng Burp Suite bắt lại gói tin có Credential Keycloak (được mã hóa) và gửi lại cho Keycloak -> Báo 302 (*Found*) nhưng không dẫn đi đâu tiếp và trên ELK hiện log *LOGIN_ERROR* từ Keycloak kèm IP của Attacker!

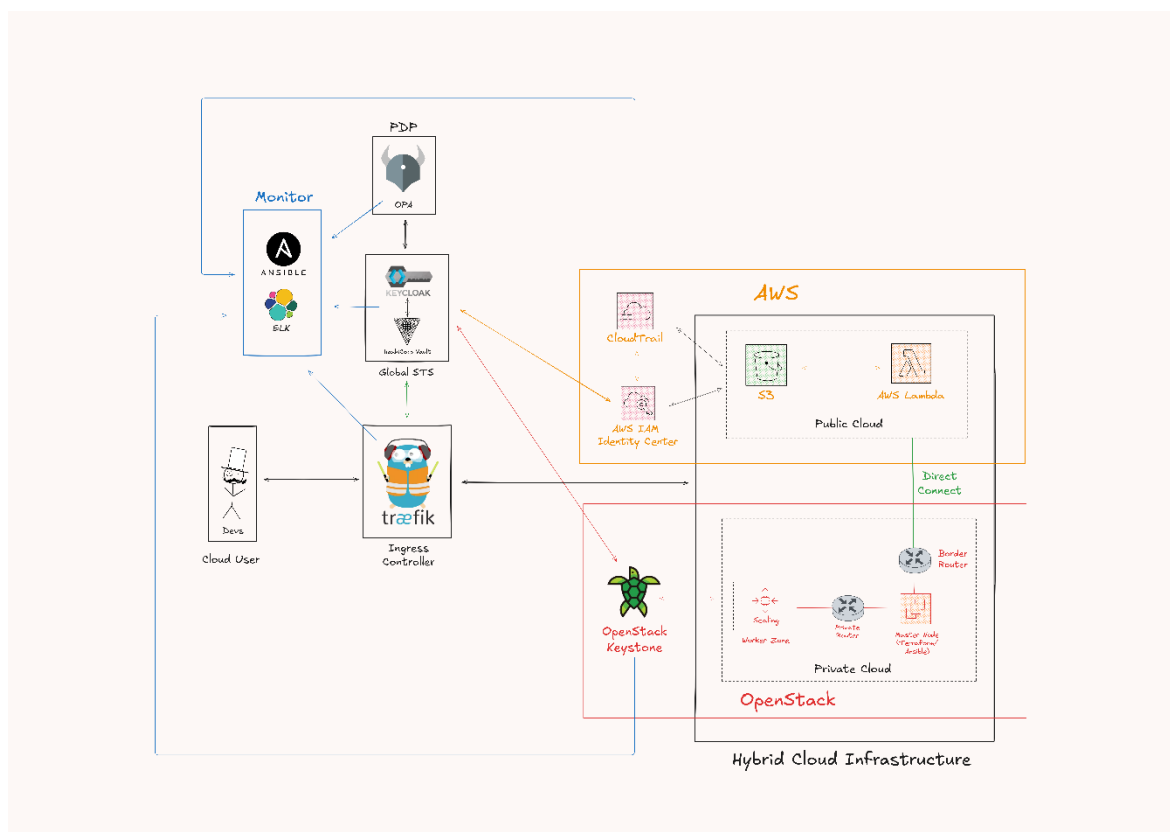
Demo:

<https://drive.google.com/file/d/1vVZVvKvNL7cYqv7rAnJDKhQS3Po2zYEWhs/view?usp=sharing>

4.6. Nhận xét

- Hệ thống đã phối hợp thành công các Identity Control Plane của từng Cloud thành 1 Unified Identity Layer thông qua OIDC/SAML, đảm bảo triết lý "Unify, not merge"
- Bằng cách thiết lập Keycloak làm Identity Provider (IdP) trung tâm, dự án đã chuyển đổi mỗi quan hệ nhân ($M \times N$) thành mỗi quan hệ cộng ($M + N$). Quản trị viên giờ đây chỉ cần quản lý M định danh tại một nơi duy nhất và thiết lập N Trust Relationship với các SP.
- Logstash ETL Pipeline đã giải quyết thành công việc chuẩn hóa dữ liệu từ các nguồn không đồng nhất. Phản ứng được chuẩn hóa và tự động hóa (as-Code) bằng Ansible playbook.
- Các bài thử nghiệm (Functionality, Unsigned Token, Replay Attack, MFA) cho thấy cơ chế SSO và Federation hoạt động ổn định, có khả năng phòng thủ trước những phương thức tấn công phổ biến.

4.7. Hướng phát triển



- Tích hợp thêm HashiCorp Vault để triển khai Key Rotation và quản lý Secret tự động.
- Tích hợp Traefik/Nginx như 1 Ingress Controller, nâng cao tính chuyên nghiệp cho trải nghiệm người dùng và giải quyết những vấn đề tồn đọng (Người dùng không phải truy cập URL Federation của OpenStack và AWS riêng, traffic không phải lúc nào cũng cần qua Keycloak ...)
- Triển khai Policy tập trung bằng OPA, đóng vai trò như Policy Decision Point (PDP) cho cả hệ thống, nhất quán hóa việc kiểm soát quyền hạn trên cả Private và Public Cloud.
- Xây dựng quy trình để Git trở thành "Single Source of Truth" lưu trữ toàn bộ mã nguồn cấu hình, từ Ansible Playbook, chính sách OPA đến Infrastructure-as-Code (IaC). Mọi thay đổi trong hạ tầng sẽ được kiểm soát phiên bản và tự động triển khai.

TÀI LIỆU THAM KHẢO

- [1] Elastic, "Configure SSL, TLS, and HTTPS to secure Elasticsearch, Kibana, Beats, and Logstash," *Elastic Guide*. [Online]. Địa chỉ: [Set up transport TLS | Elastic Docs](#)
- [2] OpenStack, "Identity API v3 extensions", *Keystone documentation*. [Trực tuyến]. Địa chỉ: [Identity API v3 extensions \(CURRENT\) — keystone documentation](#)
- [3] OpenStack, "TLS — kolla-ansible 21.1.0.dev303 documentation", kolla-ansible documentation. [Trực tuyến]. Địa chỉ: [TLS — kolla-ansible 21.1.0.dev303 documentation](#)
- [4] OpenStack, "Advanced Configuration — kolla-ansible 21.1.0.dev303 documentation", kolla-ansible documentation. [Trực tuyến]. Địa chỉ: [Advanced Configuration — kolla-ansible 21.1.0.dev303 documentation](#)
- [5] Ansible, "community.general.keycloak_client module — Ansible Community Documentation", Ansible Documentation. [Trực tuyến]. Địa chỉ: [community.general.keycloak_client module – Allows administration of Keycloak clients using Keycloak API — Ansible Community Documentation](#)
- [6] Elastic, "Painless Scripting Language", Elasticsearch Guide. [Trực tuyến]. Địa chỉ: [Introduction to Painless | Elastic Docs](#)
- [7] K. M. Fox, "AWS SSO with Keycloak", GitHub. [Trực tuyến]. Địa chỉ: [kevinmfox/aws-sso-with-keycloak](#)